

Day 8 — Complete Supervised ML Lifecycle

Beginner-friendly summary

A supervised ML system is much more than:

```
model.fit(X_train, y_train)
```

A production-quality lifecycle is closer to:

```
Business problem
↓
Decision to improve
↓
Prediction target
↓
Data + labels
↓
Data contracts + quality checks
↓
Versioned dataset snapshot
↓
Leakage-safe split
↓
Baseline
↓
Preprocessing + feature pipeline
↓
Train
↓
Validate + choose threshold
↓
Test once
↓
Document model/data/risk
↓
Deploy
↓
Measure online business impact
↓
Monitor → retrain / rollback / retire
```

The senior-level idea is:

A good ML model is not necessarily a good ML system.

A model can have excellent offline accuracy and still be useless because:

- the target does not correspond to a useful business decision;
- labels are delayed or wrong;
- future information leaked into training;

- preprocessing differs between training and production;
- the data changes;
- there is no usable intervention;
- the model merely automates something deterministic rules could do better;
- the chosen metric does not correspond to business value;
- nobody can reproduce the model six months later.

For Day 8, use one mental model throughout:

At invoice creation time, predict whether an invoice is likely to become a payment exception within the next 30 days, so finance operations can prioritize proactive review.

This is hypothetical and used only as a learning example.

1. Start with the Business Decision, Not the Algorithm

Suppose Finance says:

"We want AI to reduce invoice exceptions."

That is not yet an ML problem.

We need to decompose it.

Business objective

Reduce costly payment exceptions.

Decision

Which newly received invoices should receive additional review?

Prediction

For invoice i at time t :

$P(\text{invoice becomes an exception within 30 days} \mid \text{information known at } t)$

Intervention

Possible action:

```

High predicted risk
    ↓
Route invoice to proactive review
    ↓
Check vendor/details/approval/documentation
  
```

Outcome

Did the invoice actually experience an exception during the next 30 days?

That gives the target:

$y = 1 \rightarrow$ exception occurred within 30 days
 $y = 0 \rightarrow$ no exception occurred within 30 days

But notice the distinction:

Prediction $\neq$ Decision $\neq$ Intervention $\neq$ Business outcome

The model predicts.

A policy decides what to do with that prediction.

The intervention changes the business process.

The business metric tells us whether that process helped.

2. The Four Questions I Expect Before Modeling

At senior level, clarify four things immediately.

Question	Finance example
What are we predicting?	30-day payment exception
When is prediction made?	Invoice creation/approval time
What decision changes?	Prioritize proactive review
What proves success?	Lower operational loss/delay at acceptable review cost

This prevents a common failure:

"We achieved 93% accuracy."

So what?

Did anyone act differently?

Did exceptions decrease?

Was there enough reviewer capacity?

Did the intervention actually work?

3. ML Problem Types

Supervised learning

You have:

$X \rightarrow$ known features
 $y \rightarrow$ known labels

Examples:

- fraud prediction
- churn prediction
- default prediction
- invoice exception prediction
- demand forecasting
- transaction classification

Classification:

invoice → exception / no exception

Regression:

customer features → expected lifetime value

Unsupervised learning

You have observations but no authoritative target labels.

Examples:

- customer segmentation
- transaction clustering
- embedding-based grouping
- dimensionality reduction
- exploratory pattern discovery

Conceptually:

X
↓
discover structure

rather than:

$X \rightarrow y$

Semi-supervised learning

You have:

small amount of labeled data
+
large amount of unlabeled data

Potential techniques include:

- pseudo-labeling
- self-training
- representation learning followed by supervised fine-tuning
- label propagation

Useful when labeling is expensive.

But pseudo-labels are **not automatically ground truth**.

Bad predictions can reinforce themselves.

4. Anomaly Detection Is Slightly Different

Suppose Finance asks:

| Which transactions look unusual?

There may be no reliable label saying:

transaction 38291 = anomaly

You may therefore model normal behavior and find deviations.

Examples:

unusually large transaction
unexpected vendor
unusual posting time
unexpected account combination
abnormal expense pattern

Possible methods include:

- statistical thresholds
- Isolation Forest
- One-Class SVM
- autoencoders
- density-based methods
- rules

An anomaly is not necessarily fraud.

Anomalous ≠ fraudulent

It means unusual according to the model's definition.

5. When ML Is the Wrong Tool

One of the strongest senior-level decisions is sometimes:

Don't use ML.

Suppose policy says:

```
if invoice_amount > ₹10,000,000:  
    require CFO approval
```

Do not train an ML model to learn that rule.

Use deterministic logic.

Why?

Because the requirement is:

- explicit;
- auditable;
- stable;
- deterministic;
- legally or operationally enforceable.

A useful separation is:

Requirement	Preferred approach
Exact policy	Rules
Arithmetic	Deterministic code
Known validation	Schema/rules
Predict uncertain future outcome	ML
Discover unusual behavior	Anomaly detection
Discover natural groups	Unsupervised ML

6. Labels Are Often Harder Than Features

Consider:

```
prediction time = January 1  
  
target =  
"Did this invoice have an exception during the next 30 days?"
```

On January 2, you don't know the label.

You must wait.

That is **label delay**.

Training-data cutoff

Suppose today were August 1 and your label horizon were 30 days.

Training examples from July 20 may not have mature labels yet.

You cannot safely say:

```
July 20 invoice → no exception
```

because its 30-day observation period has not finished.

Instead:

```
training cutoff ≤ current_date - label_window
```

This is often overlooked.

7. Ground Truth

"Ground truth" sounds absolute.

In reality, labels often come from imperfect processes.

Examples:

```
fraud label ← investigator determination  
customer churn ← account status  
invoice error ← downstream correction  
disease label ← medical diagnosis  
spam label ← human annotation
```

Each may contain errors.

Ask:

| What real-world process created y ?

That question often reveals more risk than model selection does.

8. Weak and Noisy Labels

Suppose we don't have a human-confirmed invoice exception label.

Someone creates:

```
exception = late_payment_days > 10
```

This might be useful.

But it is a **proxy**.

Maybe invoices were late because:

- banking systems failed;
- a supplier changed bank details;
- a holiday delayed processing;
- internal approval was delayed.

The proxy doesn't necessarily represent the actual business phenomenon.

A strong model trained on poor labels becomes:

| very good at reproducing the poor label definition.

9. Feedback Loops

This deserves particular attention in senior ML systems.

Imagine:

```

model predicts high risk
      ↓
invoice receives extra review
      ↓
review prevents exception
      ↓
observed target = no exception
  
```

Now your training data says:

```

high-risk invoice → no exception
  
```

But that may have happened **because the model caused an intervention.**

You now have:

```

prediction
      ↓
decision
      ↓
changes future data
      ↓
becomes next training dataset
  
```

This is a feedback loop.

You need to track:

- model score;
- intervention;
- treatment/control status where appropriate;
- final outcome;
- human override.

Otherwise the model can corrupt its own future training labels.

10. Data Contracts

Before modeling, establish what data is allowed to look like.

Example contract:

```
invoice_id
  type: string
  nullable: false
  unique: true

invoice_date
  type: datetime
  nullable: false

invoice_amount
  type: float
  nullable: false
  range: > 0

department
  type: category
  allowed:
    Finance
    Sales
    Engineering
    Operations

payment_terms_days
  type: integer
  allowed: 1..180

exception_30d
  type: integer
  allowed: {0, 1}
```

This creates a boundary between:

```
upstream producer
  ↓
data contract
  ↓
ML consumer
```

Without one, upstream changes can silently corrupt ML.

11. Schema Validation vs Data-Quality Validation

They are related but different.

Schema

Asks:

```
Does column invoice_amount exist?  
  
Is it numeric?  
  
Is department a string?
```

Data quality

Asks:

```
Are 70% of invoice amounts suddenly missing?  
  
Did median amount change by 500%?  
  
Did a new category appear?  
  
Did invoice IDs become duplicated?  
  
Did today's dataset shrink unexpectedly?
```

The schema could remain valid while the data becomes useless.

12. Important Data-Quality Dimensions

A good senior vocabulary is:

Completeness

Are required values present?

```
null rate  
missing records
```

Validity

Do values satisfy rules?

```
amount > 0  
currency ∈ allowed set
```

Uniqueness

```
invoice_id must be unique
```

Consistency

Does the same concept use consistent definitions?

```
"Engineering"  
"ENG"  
"engineering"
```

may actually represent one category.

Timeliness

Did data arrive when expected?

Accuracy

Does the stored value represent reality?

Integrity

Do relationships make sense?

```
vendor_id exists in vendor master
```

Distribution stability

Did the statistical population change?

13. Ownership Matters

Every important feature should ultimately have an owner.

For example:

```
invoice_amount  
Owner: Finance Data Platform  
  
vendor_country  
Owner: Procurement Master Data  
  
payment_status  
Owner: Accounts Payable
```

Otherwise ML teams eventually face:

| This field changed three weeks ago. We don't know why.

14. Data-Quality SLA

You may define expectations such as:

```
dataset available before scheduled training  
required columns present
```

```
duplicate key rate below approved limit
missing values below approved thresholds
referential integrity satisfied
label freshness within approved delay
```

The exact numbers must come from actual system requirements.

Do not invent arbitrary production SLAs.

15. Dataset Snapshots

This is critical for reproducibility.

Do not say:

```
We trained using the invoice table.
```

Ask:

```
| Which exact version?
```

Possible identifiers:

```
dataset name
snapshot timestamp
partition range
query version
source version
hash
schema version
```

For example:

```
finance_invoice_training
snapshot=2026-07-31
schema=v4
query=9fdc21...
sha256=a31f...
```

Then later you can answer:

```
| What exact records produced model v17?
```

16. Lineage and Provenance

Provenance

Where did this data originate?

```
SAP
↓
finance_raw
↓
clean_invoice
↓
ml_training
```

Lineage

What transformations produced the value?

```
ERP.invoice_total
↓ currency conversion
↓ tax adjustment
↓ aggregation
↓ model feature
```

This matters especially in finance, healthcare, regulated AI, and enterprise audit environments.

17. Experiment Tracking

A reproducible experiment should capture more than metrics.

At minimum:

```
experiment_id
code version
data version
feature version
environment version
model class
hyperparameters
random seed
split definition
metric definitions
threshold
evaluation results
model artifact
```

Tools can include:

- MLflow
- Vertex AI Experiments
- SageMaker Experiments
- Weights & Biases
- custom internal systems

The tool is less important than the discipline.

18. Train / Validation / Test

A common setup:

```
TRAIN
  learn parameters

VALIDATION
  model/hyperparameter/threshold selection

TEST
  final unbiased estimate
```

You should conceptually treat test data as unavailable until the design is frozen.

19. Why Random Splits Can Be Dangerous

For generic IID data:

```
train_test_split(..., random_state=42)
```

can be perfectly reasonable.

But enterprise datasets frequently have structure.

Stratified split

Useful when classes are imbalanced.

It attempts to preserve:

```
positive/negative proportion
```

across splits.

For example:

```
Train: 5% fraud
Validation: ~5%
Test: ~5%
```

20. Group Split

Imagine one customer has 50 transactions.

Random splitting can produce:

```
customer 123
  transaction A → train
```

```
transaction B → train
transaction C → test
```

The model may indirectly recognize that customer.

If you need performance on unseen customers:

```
customer 123 → train only
customer 456 → test only
```

Use group splitting.

Potential groups:

- customer
- patient
- vendor
- device
- household
- store
- account

21. Temporal Split

For predictions about the future:

past	—————	future
TRAIN	VALIDATION	TEST
Jan-Jun	Jul-Aug	Sep-Oct

This often better reflects deployment.

You train on history and predict later events.

22. Point-in-Time Correctness

One of today's most important ideas.

Suppose we're predicting on:

```
April 1, 10:00 AM
```

Every feature must answer:

| Was this information actually available at April 1, 10:00 AM?

If not, it cannot be used.

Imagine a feature:

```
vendor_total_exceptions_last_90_days
```

It sounds fine.

But if you compute it today using the current warehouse:

```
April 1 invoice
      ↓
feature accidentally includes
April 10 exception
```

you leaked future information.

The correct aggregate is:

```
events with timestamp < prediction_timestamp
```

This is **point-in-time correctness**.

23. Leakage Taxonomy

Target leakage

Feature effectively reveals the target.

Example:

Predict:

```
Will invoice require escalation?
```

Feature:

```
escalation_reason
```

Obviously dangerous.

Future leakage

Using information not yet available.

Example:

```
future payment status
future customer activity
post-approval data
```

Entity leakage

Related observations appear across training/test boundaries.

Examples:

```
same patient
same customer
same vendor
same machine
```

Pipeline leakage

Preprocessing learns from validation/test data.

Bad:

```
scaler.fit(all_data)
train, test = split(all_data)
```

Good:

```
train, test = split(data)

pipeline.fit(train)
pipeline.predict(test)
```

24. The Classic Scaling Leakage Error

Wrong:

```
X_scaled = scaler.fit_transform(X)

X_train, X_test = train_test_split(X_scaled)
```

The scaler already saw the test population.

Correct:

```
split
↓
fit scaler using TRAIN
↓
transform TRAIN
transform VALIDATION
transform TEST
```

Even better:

```
Pipeline([
    ("preprocessing", ...),
    ("model", ...)
])
```

Then:

```
pipeline.fit(X_train, y_train)
```

controls this automatically.

25. Data Cleaning

Data problems should not simply be "fixed" blindly.

Every cleaning decision changes your statistical population.

Missing values

Ask why the data is missing.

Three broad concepts are useful:

```
MCAR
Missing Completely At Random

MAR
Missing At Random conditional on observed information

MNAR
Missing Not At Random
```

Example:

```
income missing
```

might itself communicate risk.

Possible treatments:

- imputation;
 - explicit "missing" category;
 - missing indicator;
 - model with native missing-value support;
 - discard examples;
 - fix upstream collection.
-

26. Outliers

An outlier may be:

data error
or
rare legitimate event

Invoice amount:

₹200 trillion

probably data corruption.

But:

₹100 million

might be a legitimate enterprise transaction.

Blind removal can erase exactly the cases the model needs to recognize.

27. Duplicates

Ask:

| What does duplicate mean?

Two identical rows?

Same invoice ID?

Same vendor + amount + date?

A duplicated business event may have different technical IDs.

Deduplication needs a business definition.

28. Inconsistent Categories

You may see:

Sales
sales
SALES
Sales Department
SLS

Some can be normalized.

But don't collapse categories unless domain semantics confirm equivalence.

29. Label Errors

For supervised learning:

```
quality(X) matters  
  
quality(y) often matters even more
```

You can inspect:

- suspicious high-loss examples;
 - reviewer disagreement;
 - inconsistency across annotators;
 - class-specific error patterns;
 - impossible labels;
 - duplicates with conflicting labels.
-

30. Preprocessing Pipeline

Conceptually:

```
raw record  
  |  
  +--> numeric features  
  |    missing-value handling  
  |    scaling  
  |  
  +--> categorical features  
      missing-value handling  
      encoding  
      |  
      v  
  feature matrix  
      |  
      v  
  model
```

Critical rule:

| Anything learned from data must be fitted using training data only.

This includes:

- means;
- medians;
- standard deviations;
- category vocabularies;

- target encoding;
 - feature selection;
 - PCA;
 - learned imputations;
 - transformations.
-

31. fit() vs transform()

Suppose training values are:

```
10, 20, 30
```

Median:

```
20
```

Training:

```
imputer.fit(train)
```

learns:

```
median = 20
```

Then:

```
imputer.transform(validation)  
imputer.transform(test)
```

use 20.

You do **not** calculate a different median from test data.

32. Training-Serving Consistency

Another production problem:

Training:

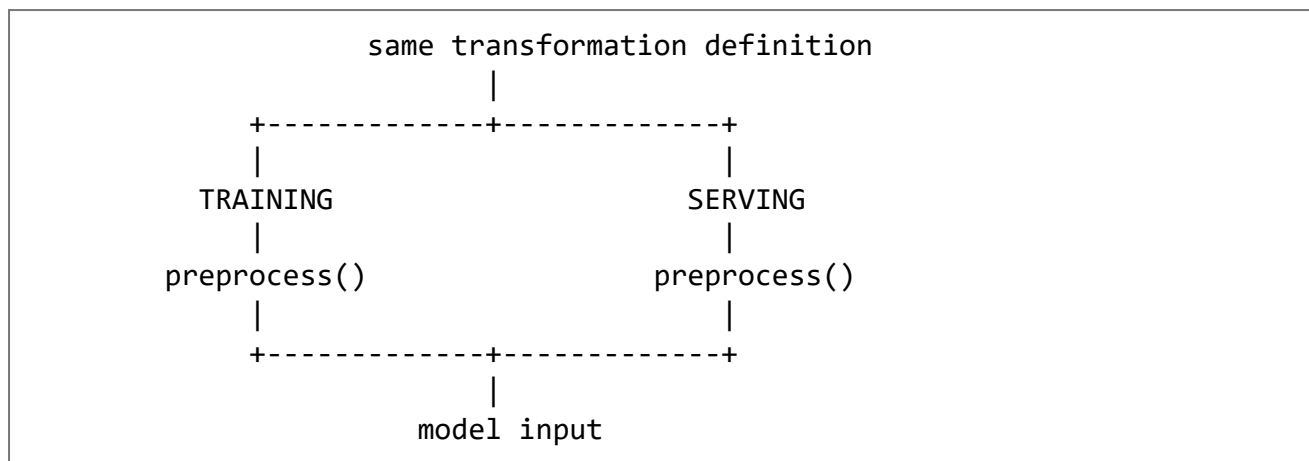
```
country = country.lower().strip()
```

Production:

```
country = incoming_country
```

Now preprocessing differs.

Better architecture:



Reuse the same pipeline, feature definitions, or feature platform whenever possible.

33. Baseline First

Before sophisticated ML, build something simple.

Examples:

Majority baseline

always predict "no exception"

Historical-rate baseline

predict global exception probability

Rules baseline

very large invoice → high risk

Simple model

logistic regression

Why?

Because otherwise:

GradientBoosting AUC = X

means almost nothing.

Maybe a trivial rule performs just as well.

34. Baselines Protect Against Complexity Theater

Imagine:

Rule: 0.70 performance
Logistic regression: 0.72
Large neural model: 0.721

Assuming those hypothetical values came from a valid evaluation, the neural model would need extraordinary operational benefits to justify:

- increased latency;
- infrastructure complexity;
- monitoring burden;
- explanation difficulty;
- maintenance;
- cost.

The best production system is not necessarily the most sophisticated model.

35. Offline Metrics

For classification you might measure:

accuracy
precision
recall
F1
ROC-AUC
PR-AUC
log loss
Brier score
calibration

Which metric matters depends on the decision.

For rare exceptions, accuracy may be terrible as the main metric.

Imagine:

99% normal
1% exception

Model:

always predict normal

Accuracy:

99%

Useful exceptions detected:

0%

36. Precision and Recall

Precision

Of the invoices we flagged:

how many really became exceptions?

$$Precision = \frac{TP}{TP + FP}$$

Recall

Of all genuine exceptions:

how many did we identify?

$$Recall = \frac{TP}{TP + FN}$$

Which matters more depends on business economics.

37. Threshold Is Part of the Model System

Suppose logistic regression returns:

0.12
0.44
0.61
0.93

The model outputs probabilities.

The decision system chooses:

score >= threshold → review

Changing the threshold changes:

- precision;
- recall;
- workload;
- false positives;
- false negatives;
- cost.

Therefore reproducibility requires:


```
model version
+
threshold version
```

not merely model weights.

38. Offline vs Online

Offline:

```
historical dataset
    ↓
model
    ↓
precision / recall / AUC
```

Online:

```
production users
    ↓
model recommendation
    ↓
business intervention
    ↓
business outcome
```

Possible online outcomes:

- number of prevented exceptions;
- review workload;
- manual handling time;
- financial loss;
- customer delay;
- override rate;
- user adoption.

Again, actual metrics and target values must come from the real business problem.

39. Why Offline Improvement May Not Produce Business Improvement

You improve:

```
PR-AUC
```

but business impact stays unchanged.

Why?

Possibilities:

```
model
  ↓
predictions
  ↓
nobody sees them

or

reviewers ignore them

or

there is no capacity to review

or

predictions arrive too late

or

action cannot change the outcome

or

false positives overwhelm operations
```

This gives a senior-level chain:

```
Model metric
  ↓
Decision quality
  ↓
Operational behavior
  ↓
Business outcome
```

Each transition can fail.

40. Model Cards

A model card should explain the model beyond its artifact file.

Typical sections:

```
Model name/version
Purpose
Prediction target
Intended users
```

Intended use
Prohibited use
Training data
Evaluation data
Features
Algorithm
Metrics
Threshold
Known limitations
Risk considerations
Bias/fairness evaluation
Security/privacy
Monitoring
Owner
Approval
Retirement criteria

41. Data Cards

A corresponding data card might capture:

Dataset name/version
Purpose
Sources
Collection process
Population
Time range
Label definition
Label delay
Sampling
Exclusions
Known quality issues
Missing-data behavior
Sensitive attributes
Lineage
Licensing/usage restrictions
Owner
Retention

42. Intended and Prohibited Use

Suppose our model predicts payment exceptions.

Intended

prioritize invoices for operational review
--

Prohibited

automatically accuse a supplier of fraud
automatically block payment without an approved control
use the score for unrelated employee evaluation

A model should not silently expand into business decisions it was never evaluated for.

43. Risk Classification

Not every model needs the same governance.

Conceptually:

LOW RISK
internal recommendation
non-consequential

MEDIUM RISK
financial workflow prioritization

HIGH RISK
credit decision
employment
healthcare
legal/regulatory consequence

Governance intensity should scale with potential harm.

44. Reproducibility Is a Chain

To reconstruct a prediction system, you may need:

Code
+
Data
+
Labels
+
Feature definitions
+
Preprocessing
+
Environment
+
Model parameters
+
Random seed

```
+  
Threshold  
+  
Evaluation code  
=  
Reproducible experiment
```

A powerful senior answer is:

"The model weights alone are not the reproducible unit. The reproducible unit is the complete experiment specification."

Practical Task

We'll now build a small reproducible finance ML project.

Problem

Predict:

| Whether an invoice will become an operational exception within 30 days.

Prediction occurs at invoice creation time.

Features will deliberately contain only information assumed available then.

Design Reasoning Before Code

I would structure this exercise around these invariants.

Invariant 1 — No future information

Every feature must exist when the invoice arrives.

Therefore we deliberately avoid:

```
final_payment_date  
eventual_payment_status  
future_approval_result  
actual_days_late  
exception_reason
```

because those reveal future events.

Invariant 2 — Time flows forward

Because deployment predicts future invoices:

```
older → train  
newer → validation  
newest → test
```

rather than random splitting.

Invariant 3 — All learned preprocessing belongs inside the pipeline

```
imputation
encoding
scaling
model
```

are fit on training only.

Invariant 4 — Baseline before candidate model

We'll evaluate:

```
DummyClassifier
rule baseline
logistic regression
```

before considering more complex models.

Invariant 5 — Validation selects threshold

The test set must not choose the operating threshold.

```
train → learn weights

validation → choose threshold

test → final evaluation
```

Invariant 6 — Every artifact has an identity

We'll record:

- dataset hash;
 - environment versions;
 - feature list;
 - split boundaries;
 - seed;
 - experiment ID;
 - threshold;
 - model artifact hash.
-

Pseudocode

```
SET deterministic random seed

GENERATE synthetic invoices
  invoice metadata
  numeric features
  categorical features
  binary target

VALIDATE data contract
  columns
  nullability
  ranges
  allowed categories
  uniqueness
  target domain

CREATE reproducible dataset snapshot
CALCULATE dataset hash

SORT records chronologically

SPLIT
  oldest 70% → train
  next 15% → validation
  newest 15% → test

DEFINE feature lists

BUILD preprocessing
  numeric:
    median imputation
    scaling

  categorical:
    most-frequent imputation
    one-hot encoding

BUILD baseline
  DummyClassifier

BUILD rules baseline

BUILD candidate
  preprocessing
  logistic regression

FIT candidate on TRAIN ONLY

PREDICT validation probabilities
```

```
SELECT threshold from validation
    according to explicit business operating requirement

FREEZE threshold

EVALUATE test once

SAVE
    pipeline artifact
    model hash
    metrics
    manifest
    experiment record

DOCUMENT
    model card
    data card
```

Complete scikit-learn Reference Implementation

```
from __future__ import annotations

import hashlib
import json
import os
import platform
import uuid
from datetime import datetime, timezone
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import sklearn
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```



```

# -----
# Configuration
# -----

SEED = 42
N_ROWS = 8_000

ARTIFACT_DIR = Path("artifacts")
ARTIFACT_DIR.mkdir(exist_ok=True)

TARGET = "exception_30d"

NUMERIC_FEATURES = [
    "invoice_amount",
    "payment_terms_days",
    "vendor_tenure_months",
    "is_month_end",
]

CATEGORICAL_FEATURES = [
    "department",
    "approval_level",
    "country",
]

FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES

# -----
# Synthetic dataset
# -----

def make_synthetic_finance_data(
    n_rows: int = N_ROWS,
    seed: int = SEED,
) -> pd.DataFrame:
    rng = np.random.default_rng(seed)

    start = pd.Timestamp("2024-01-01")

    event_dates = start + pd.to_timedelta(
        rng.integers(0, 730, size=n_rows),
        unit="D",
    )

    departments = rng.choice(
        ["Finance", "Sales", "Engineering", "Operations"],
        size=n_rows,
        p=[0.20, 0.25, 0.30, 0.25],
    )

```

```

)

approval_levels = rng.choice(
    ["manager", "director", "vp"],
    size=n_rows,
    p=[0.70, 0.25, 0.05],
)

countries = rng.choice(
    ["IN", "US", "GB", "DE"],
    size=n_rows,
    p=[0.45, 0.25, 0.15, 0.15],
)

invoice_amount = rng.lognormal(
    mean=8.2,
    sigma=1.0,
    size=n_rows,
)

payment_terms_days = rng.choice(
    [15, 30, 45, 60, 90],
    size=n_rows,
    p=[0.10, 0.45, 0.20, 0.20, 0.05],
)

vendor_tenure_months = rng.integers(
    1,
    121,
    size=n_rows,
)

is_month_end = (
    pd.Series(event_dates).dt.day >= 25
).astype(int).to_numpy()

# Synthetic hidden data-generating mechanism.
# It is used only to generate labels for the learning exercise.
logit = (
    -4.5
    + 0.30 * np.log1p(invoice_amount)
    + 0.50 * (payment_terms_days >= 60)
    + 0.50 * (vendor_tenure_months < 6)
    + 0.35 * (approval_levels == "vp")
    + 0.25 * (departments == "Operations")
    + 0.20 * is_month_end
)

probability = 1 / (1 + np.exp(-logit))

exception_30d = rng.binomial(

```

```

        n=1,
        p=probability,
    )

df = pd.DataFrame(
    {
        "invoice_id": [
            f"INV-{i:07d}"
            for i in range(n_rows)
        ],
        "event_date": event_dates,
        "invoice_amount": invoice_amount.round(2),
        "payment_terms_days": payment_terms_days,
        "vendor_tenure_months": vendor_tenure_months,
        "department": departments,
        "approval_level": approval_levels,
        "country": countries,
        "is_month_end": is_month_end,
        TARGET: exception_30d,
    }
)

# Insert a small amount of missing feature data intentionally.
# The target and identifiers remain complete.
missing_idx = rng.choice(
    df.index,
    size=max(1, int(0.01 * len(df))),
    replace=False,
)

df.loc[missing_idx, "vendor_tenure_months"] = np.nan

return df.sort_values(
    ["event_date", "invoice_id"]
).reset_index(drop=True)

# -----
# Data contract
# -----

DATA_CONTRACT = {
    "invoice_id": {
        "nullable": False,
        "unique": True,
    },
    "event_date": {
        "nullable": False,
    },
    "invoice_amount": {
        "nullable": False,
    },

```

```

        "min_exclusive": 0,
    },
    "payment_terms_days": {
        "nullable": False,
        "allowed": {15, 30, 45, 60, 90},
    },
    "vendor_tenure_months": {
        "nullable": True,
        "min_inclusive": 0,
    },
    "department": {
        "nullable": False,
        "allowed": {
            "Finance",
            "Sales",
            "Engineering",
            "Operations",
        },
    },
    "approval_level": {
        "nullable": False,
        "allowed": {
            "manager",
            "director",
            "vp",
        },
    },
    "country": {
        "nullable": False,
        "allowed": {
            "IN",
            "US",
            "GB",
            "DE",
        },
    },
    "is_month_end": {
        "nullable": False,
        "allowed": {0, 1},
    },
    TARGET: {
        "nullable": False,
        "allowed": {0, 1},
    },
}

```

```

# -----
# Data-quality validation
# -----

```

```

def validate_data(
    df: pd.DataFrame,
    contract: dict,
) -> None:
    errors: list[str] = []

    missing_columns = (
        set(contract.keys()) - set(df.columns)
    )

    if missing_columns:
        errors.append(
            f"Missing columns: {sorted(missing_columns)}"
        )

    if errors:
        raise ValueError("\n".join(errors))

    for column, rules in contract.items():

        if not rules.get("nullable", True):
            null_count = df[column].isna().sum()

            if null_count:
                errors.append(
                    f"{column}: "
                    f"{null_count} unexpected null values"
                )

        if rules.get("unique"):
            duplicate_count = df[column].duplicated().sum()

            if duplicate_count:
                errors.append(
                    f"{column}: "
                    f"{duplicate_count} duplicate values"
                )

        allowed = rules.get("allowed")

        if allowed is not None:
            observed = set(
                df[column].dropna().unique()
            )

            unexpected = observed - allowed

            if unexpected:
                errors.append(
                    f"{column}: unexpected values "
                    f"{sorted(unexpected)}"
                )

```

```

        )

    if "min_exclusive" in rules:
        invalid = (
            df[column].dropna()
            <= rules["min_exclusive"]
        )

        if invalid.any():
            errors.append(
                f"{column}: values must be > "
                f"{rules['min_exclusive']}"
            )

    if "min_inclusive" in rules:
        invalid = (
            df[column].dropna()
            < rules["min_inclusive"]
        )

        if invalid.any():
            errors.append(
                f"{column}: values must be >= "
                f"{rules['min_inclusive']}"
            )

    if not pd.api.types.is_datetime64_any_dtype(
        df["event_date"]
    ):
        errors.append(
            "event_date must be datetime64"
        )

    if errors:
        raise ValueError(
            "Data-quality validation failed:\n- "
            + "\n- ".join(errors)
        )

# -----
# Dataset version
# -----

def dataframe_sha256(df: pd.DataFrame) -> str:
    stable = df.sort_values(
        "invoice_id"
    ).reset_index(drop=True)

    csv_bytes = stable.to_csv(
        index=False,

```

```

        date_format="%Y-%m-%dT%H:%M:%S",
    ).encode("utf-8")

    return hashlib.sha256(
        csv_bytes
    ).hexdigest()

def file_sha256(path: Path) -> str:
    digest = hashlib.sha256()

    with path.open("rb") as file:
        for chunk in iter(
            lambda: file.read(1024 * 1024),
            b"",
        ):
            digest.update(chunk)

    return digest.hexdigest()

# -----
# Temporal split
# -----

def temporal_split(
    df: pd.DataFrame,
    train_fraction: float = 0.70,
    validation_fraction: float = 0.15,
):
    ordered = df.sort_values(
        ["event_date", "invoice_id"]
    ).reset_index(drop=True)

    n = len(ordered)

    train_end = int(n * train_fraction)

    validation_end = int(
        n * (
            train_fraction
            + validation_fraction
        )
    )

    train = ordered.iloc[:train_end].copy()

    validation = ordered.iloc[
        train_end:validation_end
    ].copy()

```

```

test = ordered.iloc[
    validation_end:
].copy()

assert (
    train["event_date"].max()
    <= validation["event_date"].min()
)

assert (
    validation["event_date"].max()
    <= test["event_date"].min()
)

return train, validation, test

# -----
# Preprocessing/model pipeline
# -----

def build_candidate_pipeline() -> Pipeline:

    numeric_pipeline = Pipeline(
        steps=[
            (
                "imputer",
                SimpleImputer(
                    strategy="median",
                    add_indicator=True,
                ),
            ),
            (
                "scaler",
                StandardScaler(),
            ),
        ]
    )

    categorical_pipeline = Pipeline(
        steps=[
            (
                "imputer",
                SimpleImputer(
                    strategy="most_frequent",
                ),
            ),
            (
                "encoder",
                OneHotEncoder(
                    handle_unknown="ignore",

```



```

        ),
    ),
]
)

preprocessing = ColumnTransformer(
    transformers=[
        (
            "numeric",
            numeric_pipeline,
            NUMERIC_FEATURES,
        ),
        (
            "categorical",
            categorical_pipeline,
            CATEGORICAL_FEATURES,
        ),
    ]
)

model = LogisticRegression(
    max_iter=1_000,
    random_state=SEED,
)

return Pipeline(
    steps=[
        ("preprocessing", preprocessing),
        ("model", model),
    ]
)

# -----
# Evaluation
# -----

def evaluate_probabilities(
    y_true: pd.Series,
    probabilities: np.ndarray,
    threshold: float,
) -> dict[str, float]:

    predictions = (
        probabilities >= threshold
    ).astype(int)

    return {
        "accuracy": float(
            accuracy_score(
                y_true,

```

```

        predictions,
    )
),
"precision": float(
    precision_score(
        y_true,
        predictions,
        zero_division=0,
    )
),
"recall": float(
    recall_score(
        y_true,
        predictions,
        zero_division=0,
    )
),
"f1": float(
    f1_score(
        y_true,
        predictions,
        zero_division=0,
    )
),
"roc_auc": float(
    roc_auc_score(
        y_true,
        probabilities,
    )
),
"average_precision": float(
    average_precision_score(
        y_true,
        probabilities,
    )
),
}

```

```

# -----
# Threshold selection
# -----

```

```

def select_threshold_for_recall(
    y_true: pd.Series,
    probabilities: np.ndarray,
    minimum_recall: float,
) -> float:
    """
    Demonstration only.

```

In a real project, minimum_recall must come from business cost/capacity requirements rather than being invented by the ML engineer.

```
precision, recall, thresholds = (
    precision_recall_curve(
        y_true,
        probabilities,
    )
)

candidates = []

for i, threshold in enumerate(thresholds):
    if recall[i] >= minimum_recall:
        candidates.append(
            (
                precision[i],
                threshold,
            )
        )

if not candidates:
    return 0.5

# Of thresholds meeting recall requirement,
# choose the one providing highest precision.
best_precision, best_threshold = max(
    candidates,
    key=lambda item: item[0],
)

return float(best_threshold)
```

```
# -----
# Rule baseline
# -----
```

```
def rule_baseline(
    train: pd.DataFrame,
    evaluation_df: pd.DataFrame,
) -> np.ndarray:
    """
    Simple illustrative business-rule baseline.

    The amount threshold is learned from TRAIN only.
    """

    large_invoice_threshold = (
```

```

        train["invoice_amount"].quantile(0.90)
    )

    prediction = (
        (
            evaluation_df["invoice_amount"]
            >= large_invoice_threshold
        )
        |
        (
            evaluation_df["vendor_tenure_months"]
            .fillna(np.inf)
            < 6
        )
    )

    return prediction.astype(int).to_numpy()

def evaluate_hard_predictions(
    y_true: pd.Series,
    predictions: np.ndarray,
) -> dict[str, float]:

    return {
        "accuracy": float(
            accuracy_score(
                y_true,
                predictions,
            )
        ),
        "precision": float(
            precision_score(
                y_true,
                predictions,
                zero_division=0,
            )
        ),
        "recall": float(
            recall_score(
                y_true,
                predictions,
                zero_division=0,
            )
        ),
        "f1": float(
            f1_score(
                y_true,
                predictions,
                zero_division=0,
            )
        )
    }

```

```

    ),
}

# -----
# Main experiment
# -----

def run_experiment() -> None:

    experiment_id = str(uuid.uuid4())

    df = make_synthetic_finance_data()

    validate_data(
        df,
        DATA_CONTRACT,
    )

    dataset_hash = dataframe_sha256(df)

    train, validation, test = temporal_split(df)

    X_train = train[FEATURES]
    y_train = train[TARGET]

    X_validation = validation[FEATURES]
    y_validation = validation[TARGET]

    X_test = test[FEATURES]
    y_test = test[TARGET]

    # -----
    # Dummy baseline
    # -----

    dummy = DummyClassifier(
        strategy="prior",
        random_state=SEED,
    )

    dummy.fit(
        X_train,
        y_train,
    )

    dummy_probabilities = (
        dummy.predict_proba(X_test)[: , 1]
    )

    dummy_metrics = evaluate_probabilities(

```

```

        y_test,
        dummy_probabilities,
        threshold=0.5,
    )

# -----
# Rules baseline
# -----

rule_predictions = rule_baseline(
    train,
    test,
)

rule_metrics = evaluate_hard_predictions(
    y_test,
    rule_predictions,
)

# -----
# Candidate model
# -----

pipeline = build_candidate_pipeline()

pipeline.fit(
    X_train,
    y_train,
)

validation_probabilities = (
    pipeline.predict_proba(
        X_validation
    )[:, 1]
)

# Illustrative only.
# Replace with real business operating requirement.
illustrative_minimum_recall = 0.70

threshold = select_threshold_for_recall(
    y_validation,
    validation_probabilities,
    minimum_recall=(
        illustrative_minimum_recall
    ),
)

test_probabilities = (
    pipeline.predict_proba(
        X_test

```

```

    )[:, 1]
)

candidate_metrics = evaluate_probabilities(
    y_test,
    test_probabilities,
    threshold=threshold,
)

# -----
# Save model
# -----

model_path = (
    ARTIFACT_DIR
    / f"model_{experiment_id}.joblib"
)

joblib.dump(
    pipeline,
    model_path,
)

model_hash = file_sha256(model_path)

# -----
# Version manifest
# -----

manifest = {
    "experiment_id": experiment_id,
    "created_at_utc": datetime.now(
        timezone.utc
    ).isoformat(),
    "seed": SEED,
    "target": TARGET,
    "features": FEATURES,
    "dataset": {
        "type": "synthetic_finance",
        "rows": len(df),
        "sha256": dataset_hash,
    },
    "split": {
        "strategy": "temporal",
        "train_rows": len(train),
        "validation_rows": len(validation),
        "test_rows": len(test),
        "train_start": str(
            train["event_date"].min()
        ),
        "train_end": str(

```

```

        train["event_date"].max()
    ),
    "validation_start": str(
        validation["event_date"].min()
    ),
    "validation_end": str(
        validation["event_date"].max()
    ),
    "test_start": str(
        test["event_date"].min()
    ),
    "test_end": str(
        test["event_date"].max()
    ),
},
"environment": {
    "python": platform.python_version(),
    "numpy": np.__version__,
    "pandas": pd.__version__,
    "scikit_learn": sklearn.__version__,
},
"code": {
    "git_commit": os.getenv(
        "GIT_COMMIT",
        "UNKNOWN",
    )
},
"candidate": {
    "algorithm": "LogisticRegression",
    "threshold": threshold,
    "model_artifact": str(model_path),
    "model_sha256": model_hash,
},
}

```

```

manifest_path = (
    ARTIFACT_DIR
    / f"manifest_{experiment_id}.json"
)

```

```

manifest_path.write_text(
    json.dumps(
        manifest,
        indent=2,
    )
)

```

```

# -----
# Experiment log
# -----

```



```

experiment_record = {
    "experiment_id": experiment_id,
    "dummy_baseline": dummy_metrics,
    "rule_baseline": rule_metrics,
    "candidate_model": candidate_metrics,
    "threshold": threshold,
    "dataset_sha256": dataset_hash,
    "model_sha256": model_hash,
}

log_path = (
    ARTIFACT_DIR
    / "experiments.jsonl"
)

with log_path.open(
    "a",
    encoding="utf-8",
) as file:
    file.write(
        json.dumps(experiment_record)
        + "\n"
    )

print(
    json.dumps(
        experiment_record,
        indent=2,
    )
)

if __name__ == "__main__":
    run_experiment()

```

What Makes This Leakage-Safe?

Several details matter.

1. We split before fitting preprocessing

We don't do:

```

imputer.fit(df)
scaler.fit(df)

```

before splitting.

Instead:

```
pipeline.fit(X_train, y_train)
```

Therefore:

```
median  
standard deviation  
category vocabulary  
logistic-regression coefficients
```

are learned from training only.

2. Time only flows forward

```
TRAIN  
oldest records  
  
    ↓  
  
VALIDATION  
  
    ↓  
  
TEST  
newest records
```

The candidate model cannot learn from later test observations.

3. Threshold Comes From Validation

We do:

```
TRAIN  
↓  
model  
  
VALIDATION  
↓  
threshold selection  
  
TEST  
↓  
final evaluation
```

Not:

```
TEST  
↓  
keep changing threshold
```

```
↓  
report "test" result
```

The latter turns the test set into another validation set.

Important Caveat About the 30-Day Label

Our synthetic example generates labels immediately because this is a learning exercise.

A real system would enforce:

```
prediction_timestamp  
    +  
30-day observation window  
    =  
earliest label maturity
```

So your training-data builder should contain something conceptually like:

```
eligible = (  
    event_date  
    <= as_of_date - pd.Timedelta(days=30)  
)
```

Without that, recent negatives might actually be unlabeled positives.

That is a subtle but important lifecycle issue.

Why the Pipeline Uses `handle_unknown="ignore"`

Suppose training contains:

```
IN  
US  
GB  
DE
```

and production later sees:

```
SG
```

Without an unknown-category policy, inference could crash.

```
OneHotEncoder(handle_unknown="ignore")
```

prevents that particular failure.

But it does **not** mean the data issue should be ignored operationally.

A monitoring system should still report:

```
new country category detected: SG
```

because schema evolution or population change may require retraining.

Why We Added Missing Indicators

For numeric fields:

```
SimpleImputer(  
    strategy="median",  
    add_indicator=True  
)
```

does two things.

Suppose:

```
vendor_tenure = missing
```

We replace the numeric value with the training median.

But we also give the model a feature equivalent to:

```
vendor_tenure_was_missing = 1
```

This matters when missingness itself contains information.

The Rule Baseline Has an Important Leakage Detail

Notice:

```
large_invoice_threshold = (  
    train["invoice_amount"].quantile(0.90)  
)
```

We calculate the threshold using **train**.

Not:

```
df["invoice_amount"].quantile(0.90)
```

Using the full dataset would leak distribution information from validation/test.

Even something as innocent as a percentile can leak.

Production Data-Quality Checks I'd Add

The example performs structural checks.

Production would normally add statistical checks such as:

```
row count change
null-rate change
positive-label rate
mean/median/quantiles
category-frequency changes
unknown category rate
duplicate rate
feature distribution drift
label delay/freshness
event-time sanity
prediction-time vs feature-time sanity
```

For example:

```
Training:
Finance department = 20%

Today's data:
Finance department = 80%
```

The schema remains valid.

But the population changed dramatically.

Data Card Outline for This Example

Dataset

Name: Synthetic Finance Invoice Exception Dataset

Purpose: Educational dataset for demonstrating a supervised-ML lifecycle.

Source: Programmatically generated synthetic data.

Population: Hypothetical invoices.

Prediction timestamp: Invoice creation/event date.

Target:

exception_30d

indicating whether an operational exception occurs within 30 days.

Label horizon: 30 days.

Included feature categories:

- invoice amount;
- payment terms;
- vendor tenure;
- department;
- approval level;
- country;
- month-end indicator.

Excluded information:

- eventual payment status;
- final exception reason;
- future approval decisions;
- downstream settlement information.

Known limitations:

- synthetic relationships;
- does not reproduce a real finance population;
- does not model real regulatory or geographic constraints;
- contains simplified categorical distributions;
- synthetic label mechanism does not represent an actual company's exception process.

Data-quality expectations:

- unique invoice IDs;
- valid categorical values;
- positive invoice amount;
- binary target;
- valid timestamps.

Sensitive-data status: None intentionally generated in the demonstration.

Owner: Placeholder for real implementation.

Model Card Outline

Model

Name: Invoice Exception Logistic Regression Baseline

Purpose

Estimate the risk that an invoice develops an operational exception within 30 days.

Intended use

Operational prioritization for additional invoice review.

Prohibited use

Not intended to:

- make legally consequential decisions;
- accuse vendors or employees of fraud;
- autonomously reject payments;
- determine creditworthiness;
- substitute for approved financial controls.

Algorithm

Logistic Regression

with:

numeric preprocessing
+
categorical one-hot encoding

Model inputs

Only features assumed available at prediction time.

Model output

$P(\text{exception within 30 days})$

Decision threshold

Stored separately in the experiment/version manifest.

The production threshold must be selected based on actual business:

- review capacity;
- false-negative cost;
- false-positive cost;
- intervention effectiveness.

Training methodology

Temporal:

past → train
later → validation

Baselines

- historical-prior dummy model;
- deterministic rule;
- logistic regression candidate.

Evaluation

Relevant metrics may include:

- precision;
- recall;
- F1;
- ROC-AUC;
- PR-AUC;
- calibration;
- operational workload.

Actual acceptance thresholds must come from business requirements.

Known limitations

- synthetic training population;
- limited features;
- no causal estimate of review intervention;
- no real-world drift analysis;
- no fairness assessment demonstrated;
- no real operational cost function demonstrated.

Monitoring

Production monitoring should cover:

```
input quality
feature drift
prediction drift
label drift
calibration
precision/recall after label maturity
latency
failures
review workload
human overrides
business outcomes
```

Version Manifest

Our example captures the idea:


```
{
  "experiment_id": "...",
  "dataset": {
    "sha256": "..."
  },
  "environment": {
    "python": "...",
    "numpy": "...",
    "pandas": "...",
    "scikit_learn": "..."
  },
  "code": {
    "git_commit": "..."
  },
  "features": [
    "invoice_amount",
    "payment_terms_days",
    "vendor_tenure_months"
  ],
  "split": {
    "strategy": "temporal"
  },
  "candidate": {
    "algorithm": "LogisticRegression",
    "threshold": "...",
    "model_sha256": "..."
  }
}
```

This is much stronger than:

```
model_final_v2_really_final.joblib
```

The Full Reproducibility Identity

Think of a deployed prediction as being produced by:

$$\begin{aligned}
 \textit{Prediction} = & \\
 & f(\\
 & \textit{DataVersion}, \\
 & \textit{FeatureVersion}, \\
 & \textit{CodeVersion}, \\
 & \textit{EnvironmentVersion}, \\
 & \textit{ModelVersion}, \\
 & \textit{ThresholdVersion} \\
 &)
 \end{aligned}$$

I would add evaluation version as well:

Experiment

- code:v17
- dataset:v8
- schema:v4
- labels:v6
- features:v12
- environment:v5
- model:v23
- threshold:v7
- evaluation:v9

If any of those changes, you may have a different experiment.

Senior-Level Failure Modes

Failure 1 — Wrong target

You optimize:

```
payment delayed > 5 days
```

when Finance actually cares about:

```
financial loss from preventable exceptions
```

The model succeeds technically but solves the wrong problem.

Mitigation

Start with:

```
business decision  
→ intervention  
→ outcome  
→ prediction target
```

Failure 2 — No actionable intervention

Suppose you predict an exception five seconds before it happens.

Excellent model.

Zero practical usefulness.

A prediction is valuable only when:

```
prediction lead time  
>
```

time required for useful action

Failure 3 — Label leakage

Feature:

exception_resolution_status

predicts:

exception

Offline performance becomes spectacular.

Production performance collapses.

Failure 4 — Temporal leakage

Historical customer profile accidentally includes future transactions.

Solution:

feature_time <= prediction_time

for every feature.

Failure 5 — Random split hides temporal degradation

Random train/test data comes from the same time distribution.

Production encounters a new regime.

Performance drops.

Temporal evaluation can reveal this earlier.

Failure 6 — Same Entity in Train and Test

Train:

Vendor A invoices 1–500

Test:

Vendor A invoices 501–550

Model may partly memorize vendor behavior.

If your requirement is generalization to unseen vendors, use a group holdout.

The correct split depends on the deployment question.

Failure 7 — Over-cleaning

A large legitimate invoice looks like an outlier.

You remove it.

Production then sees large invoices and behaves unpredictably.

Cleaning should follow semantic reasoning, not merely z-scores.

Failure 8 — Feature Pipeline Differs Online

Training:

```
"ENGINEERING" → "engineering"
```

Serving:

```
"ENGINEERING"
```

Different input representations reach the model.

Use shared feature definitions or package preprocessing with the model.

Failure 9 — Test-Set Tuning

Engineer runs:

```
Model A → test
```

```
Model B → test
```

```
Model C → test
```

```
Model D → test
```

and picks D.

The test set is no longer independent.

It became validation data.

Failure 10 — Retraining on Unmatured Labels

Recent observations are incorrectly labeled negative simply because the 30-day outcome hasn't happened yet.

This introduces systematic label error.

Failure 11 — Feedback Loop

High-risk cases receive intervention.

The intervention prevents bad outcomes.

Retraining interprets them as naturally low-risk cases.

You need intervention/treatment information in downstream evaluation and data design.

Failure 12 — Good Model, Bad Product

Offline:

excellent recall

Production:

10× more cases than reviewers can process

The effective intervention fails.

Threshold selection must include operational capacity.

Trade-offs You Should Be Able to Explain

Decision	Benefit	Cost/risk
Random split	statistically convenient	may ignore time/entity structure
Temporal split	deployment realism	class distributions may shift
Group split	measures unseen-entity generalization	less training data/divergent distribution
Median imputation	simple/robust	can hide missingness mechanisms
One-hot encoding	interpretable/simple	dimensionality grows
Logistic regression	interpretable/strong baseline	limited nonlinear interactions
Complex model	potential predictive lift	operational/governance complexity
Higher threshold	more precision	lower recall
Lower threshold	more recall	more operational workload
Frequent retraining	adapts quickly	risk/cost/reproducibility burden

One Subtle Point: Split Strategy Answers a Business Question

Don't choose splitting mechanically.

Each split asks something different.

Random split

| Can I predict another observation from approximately the same population?

Temporal split

| Can I predict the future from the past?

Group split

| Can I generalize to previously unseen entities?

Temporal + group split

Potentially:

| Can I predict future outcomes for entities not represented in training?

That can be much harder—but sometimes much closer to the real production requirement.

What I Would Expect From a Strong Senior Applied ML Candidate

Not:

| "I'll start by trying XGBoost."

Instead:

| "First I'd define the decision we want to improve, the prediction timestamp, the actionable intervention, the outcome and label horizon. Then I'd establish point-in-time-correct features and label maturity, version the training snapshot, select a split that matches deployment—likely temporal here—and create a rules/simple-model baseline. All learned preprocessing would live inside a train-fitted pipeline. I'd use validation data for model and threshold selection and keep test data untouched until the decision is frozen. Finally I'd version data, code, environment, features, model and threshold, document intended and prohibited use, and evaluate online whether predictions actually improve the business process."

That is the difference between:

knowing ML algorithms

and:

designing an applied ML system

Day 8 Mental Model to Retain

When someone gives you an ML problem, mentally execute:

1. What decision?
2. What prediction?
3. Prediction at what exact time?
4. What intervention follows?
5. What outcome represents success?
6. Where does the label come from?
7. When does that label mature?
8. What information existed at prediction time?
9. What split reproduces deployment?
10. What is the simplest baseline?
11. What preprocessing learns from data?
12. How do we prevent leakage?
13. What does validation choose?
14. What remains untouched for final test?
15. How do we version everything?
16. How will we measure production impact?
17. What can go wrong after deployment?

If these questions are answered correctly, selecting the individual algorithm becomes only **one part** of the supervised ML lifecycle—not the lifecycle itself.

Day 8 DSA — Linked List

Topic

Linked List: pointer manipulation, fast/slow pointers, reversal, and cycle detection

For today's medium problem, we'll solve:

Reorder List — a strong linked-list interview problem because it combines **fast/slow pointers + reversal + pointer merging**.

Python is sufficient here; Go would not add much backend/concurrency value.

1. Linked List Mental Model

An array stores elements conceptually like:

```
index:  0    1    2    3
        [10] [20] [30] [40]
```

A singly linked list stores nodes connected through pointers:

```
10 → 20 → 30 → 40 → None
```

Each node contains:

```
class ListNode:
    def __init__(self, val=0, next=None):
```

```
self.val = val
self.next = next
```

Conceptually:

```
Node
+-----+-----+
| value | next  |
+-----+-----+
```

For:

```
10 → 20 → 30
```

the first node does not contain the entire list.

It contains roughly:

```
value = 10
next  = reference to node containing 20
```

2. Why Linked-List Problems Feel Difficult

Most errors happen because changing:

```
current.next
```

can destroy the only reference you had to the rest of the list.

For example, suppose:

```
1 → 2 → 3 → None
  ^
current
```

If you immediately do:

```
current.next = previous
```

without remembering node 3, you may lose the remainder of the list.

The central linked-list habit is:

Save the next pointer before changing the current pointer.

Usually:

```
next_node = current.next
current.next = previous
```



```
previous = current
current = next_node
```

3. Recognition Signals

When should linked lists come to mind?

Look for phrases such as:

- linked list
- nodes connected by next
- reverse a list
- find middle node
- cycle/loop
- remove nth node
- merge lists
- rearrange nodes
- modify list in place
- constant-extra-space requirement

More specifically:

Signal	Common technique
Find middle	Fast/slow pointers
Detect cycle	Fast/slow pointers
Reverse nodes	prev, curr, next
Remove node	Dummy node + pointers
Merge lists	Two moving pointers
Find kth from end	Two pointers with gap
Reorder list	Middle + reverse + merge

4. Core Pattern 1 — Pointer Manipulation

Suppose:

```
A → B → C → D
```

and you're currently at B.

You need to understand that:

```
node.next = something
```

changes the graph itself.

For example:

```
B.next = A
```

changes:

```
A → B → C
```

into something potentially like:

```
A → B
↑   |
|___|
```

if you don't correctly adjust other links.

That's why linked-list questions require careful pointer ordering.

5. Core Pattern 2 — Fast and Slow Pointers

Use:

```
slow = head
fast = head
```

Then:

```
slow = slow.next
fast = fast.next.next
```

So:

```
slow moves 1 step
fast moves 2 steps
```

Finding the middle

Consider:

```
1 → 2 → 3 → 4 → 5
```

Movement:

```
Start

slow
↓
1 → 2 → 3 → 4 → 5
```

↑
fast

After one iteration:

```
slow
  ↓
1 → 2 → 3 → 4 → 5
      ↑
    fast
```

After another:

```
slow
  ↓
1 → 2 → 3 → 4 → 5
                ↑
              fast
```

slow reaches the middle.

Time:

$O(n)$

Space:

$O(1)$

6. Core Pattern 3 — Reversing a Linked List

Original:

1 → 2 → 3 → None

Wanted:

3 → 2 → 1 → None

We maintain:

```
prev
curr
next_node
```

Initially:

```
prev = None

    curr
    ↓
None  1 → 2 → 3 → None
```

First iteration:

```
next_node = curr.next
curr.next = prev
prev = curr
curr = next_node
```

Now:

```
1 → None

2 → 3 → None
↑
curr

↑
prev points to 1
```

Eventually:

```
3 → 2 → 1 → None
↑
prev
```

Standard reversal template

```
def reverse(head):
    prev = None
    curr = head

    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node

    return prev
```

Memorize the **reasoning**, not merely the syntax:

```
save next
↓
```

```
reverse current link
↓
advance prev
↓
advance current
```

7. Core Pattern 4 — Cycle Detection

Imagine:

```
1 → 2 → 3 → 4 → 5
          ↑      |
          |_____|
```

A normal traversal:

```
while current:
    current = current.next
```

never reaches None.

Brute-force cycle detection

Store visited nodes:

```
seen = set()

while head:
    if head in seen:
        return True

    seen.add(head)
    head = head.next

return False
```

Complexity:

```
Time: O(n)
Space: O(n)
```

Optimized: Floyd's Cycle Detection

Use:

```
slow → 1 node/step
fast → 2 nodes/step
```

If there is a cycle, eventually they meet.

```
def has_cycle(head):
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow is fast:
            return True

    return False
```

Complexity:

```
Time: O(n)
Space: O(1)
```

Notice:

```
slow is fast
```

rather than:

```
slow.val == fast.val
```

Two different nodes can have equal values.

We need to know whether both pointers reference the **same node**.

Medium Problem — Reorder List

Problem Statement

Given:

```
L0 → L1 → L2 → ... → Ln
```

reorder it into:

```
L0 → Ln → L1 → Ln-1 → L2 → Ln-2 → ...
```

You must modify the linked list **in place**.

Example 1

Input:

```
1 → 2 → 3 → 4
```

Output:

```
1 → 4 → 2 → 3
```

Example 2

Input:

```
1 → 2 → 3 → 4 → 5
```

Output:

```
1 → 5 → 2 → 4 → 3
```

8. Recognition Signals

When seeing this problem, notice:

| We need elements alternately from the **front and back**.

If this were an array:

```
[1, 2, 3, 4, 5]
```

we could easily access:

```
front → 1  
back  → 5  
front → 2  
back  → 4
```

But a singly linked list doesn't support:

```
list[-1]
```

in $O(1)$.

That tells us we need to transform the linked list.

The key observations are:

1. Find the middle
2. Reverse the second half
3. Merge the two halves alternately

This is a classic linked-list composition pattern.

9. Brute-Force Reasoning

The simplest solution would be to put every node into an array.

For:

```
1 → 2 → 3 → 4 → 5
```

store:

```
nodes = [node1, node2, node3, node4, node5]
```

Then use:

```
left = 0  
right = n - 1
```

and reconnect:

```
nodes[0] → nodes[4]  
nodes[4] → nodes[1]  
nodes[1] → nodes[3]  
...
```

Why it works

Arrays support random access:

```
nodes[left]  
nodes[right]
```

so picking from the two ends becomes easy.

Complexity

Traversal:

```
O(n)
```

Reconnection:

```
O(n)
```

Total:

```
Time: O(n)  
Space: O(n)
```

It is correct but violates the spirit of the optimal linked-list solution because we don't need $O(n)$ additional memory.

10. Optimized Reasoning

We want:

```
Time:  O(n)
Space: O(1)
```

Break the problem into three familiar linked-list operations.

Step 1 — Find the middle

Input:

```
1 → 2 → 3 → 4 → 5
```

Use:

```
slow = head
fast = head
```

until:

```
slow
↓
3
```

We then conceptually have:

```
First:
1 → 2 → 3

Second:
4 → 5
```

Step 2 — Reverse the second half

Before:

```
4 → 5
```

After:

```
5 → 4
```

Now:

```
First half:
```

```
1 → 2 → 3
```

```
Second half:
```

```
5 → 4
```

Notice something important.

The second half is now ordered exactly how we need it:

```
last
```

```
second-last
```

```
third-last
```

```
...
```

11. Step 3 — Merge Alternately

We have:

```
first:
```

```
1 → 2 → 3
```

```
second:
```

```
5 → 4
```

Take:

```
first
```

```
second
```

```
first
```

```
second
```

Result:

```
1 → 5 → 2 → 4 → 3
```

Exactly the required ordering.

12. Pointer Diagram

Starting:

```
first
```

```
↓
```

```
1 → 2 → 3
```

```
second
```

↓
5 → 4

We want to connect:

1 → 5

but before changing 1.next, save:

```
tmp1 = first.next
```

Likewise before changing 5.next, save:

```
tmp2 = second.next
```

Then:

```
first.next = second  
second.next = tmp1
```

So:

1 → 5 → 2

Then advance:

```
first = tmp1  
second = tmp2
```

Repeat.

This is the linked-list pointer-manipulation principle again:

Save before overwriting.

13. Pseudocode

```
function reorderList(head):  
    if list contains fewer than 2 nodes:  
        return  
  
    slow = head  
    fast = head  
  
    while fast can move two steps:  
        slow = slow.next  
        fast = fast.next.next
```

```

second = slow.next
slow.next = null

reverse second half:
    prev = null
    curr = second

    while curr exists:
        next = curr.next
        curr.next = prev
        prev = curr
        curr = next

first = head
second = prev

while second exists:
    save first.next
    save second.next

    first.next = second
    second.next = saved first.next

    advance first
    advance second

```

14. Python Solution

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reorder_list(head: ListNode | None) -> None:
    if head is None or head.next is None:
        return

    # 1. Find the end of the first half.
    slow = head
    fast = head

    while fast.next and fast.next.next:
        slow = slow.next
        fast = fast.next.next

    # 2. Split and reverse the second half.
    second = slow.next
    slow.next = None
    second.next = None

```

```

prev = None
curr = second

while curr:
    next_node = curr.next
    curr.next = prev
    prev = curr
    curr = next_node

# prev is now the head of the reversed second half.
second = prev
first = head

# 3. Merge the two halves alternately.
while second:
    first_next = first.next
    second_next = second.next

    first.next = second
    second.next = first_next

    first = first_next
    second = second_next

```

The function modifies the original list, so it does not need to return a new head.

15. Dry Run

Input:

1 → 2 → 3 → 4 → 5

Find middle

slow = 3

Split:

1 → 2 → 3 → None

4 → 5 → None

Reverse second half

4 → 5

becomes:

5 → 4

Now:

```
first:
1 → 2 → 3

second:
5 → 4
```

Merge iteration 1

Before:

```
first = 1
second = 5
```

Save:

```
first_next = 2
second_next = 4
```

Reconnect:

1 → 5 → 2

Remaining:

```
first = 2
second = 4
```

Merge iteration 2

Reconnect:

2 → 4 → 3

Combined:

1 → 5 → 2 → 4 → 3

Done.

16. Why We Explicitly Split the List

This line is important:

```
slow.next = None
```

Without it, the first half remains connected to the second half.

That can make merging harder to reason about and may accidentally create cycles.

After splitting:

```
1 → 2 → 3 → None
```

```
4 → 5 → None
```

we are manipulating two clean independent lists.

17. Correctness Reasoning

We can reason about correctness in three stages.

Stage 1

Fast/slow pointers divide the list such that the second half contains at most as many nodes as the first.

For odd length:

```
1 2 3 | 4 5
```

For even length:

```
1 2 | 3 4
```

depending on the exact fast/slow condition.

Our implementation leaves the extra node, if any, in the first half.

Stage 2

Reversing the second half transforms:

```
Lmid+1 → ... → Ln
```

into:

```
Ln → Ln-1 → ...
```

which gives us the nodes in the required backward order.

Stage 3

Alternating one node from each half produces:

```
L0
Ln
L1
Ln-1
L2
Ln-2
...
```

which is exactly the required arrangement.

18. Edge Cases

Empty list

```
None
```

Do nothing.

Handled by:

```
if head is None:
    return
```

One node

```
1
```

Already correctly ordered.

Two nodes

```
1 → 2
```

Output remains:

```
1 → 2
```

Odd number of nodes

```
1 → 2 → 3 → 4 → 5
```

Output:

```
1 → 5 → 2 → 4 → 3
```


Middle node stays last.

Even number of nodes

1 → 2 → 3 → 4

Output:

1 → 4 → 2 → 3

Duplicate values

1 → 1 → 1 → 1

No issue.

We manipulate node references, not values.

19. Complexity

There are three passes.

Finding middle:

$O(n)$

Reversing half:

$O(n)$

Merging:

$O(n)$

Technically:

[$O(n) + O(n/2) + O(n/2)$]

which simplifies to:

$O(n)$

Only a fixed number of pointers are used:

slow
fast
prev
curr
next

```
first
second
```

Therefore:

 $O(1)$

extra space.

Final

Metric	Complexity
Time	$O(n)$
Extra space	$O(1)$

20. Common Mistakes

Mistake 1 — Losing the remainder of the list

Wrong:

```
curr.next = prev
curr = curr.next
```

After:

```
curr.next = prev
```

`curr.next` no longer points forward.

Correct:

```
next_node = curr.next
curr.next = prev
curr = next_node
```

Mistake 2 — Forgetting to split

Missing:

```
slow.next = None
```

can leave unwanted links and potentially create a cycle during merging.

Mistake 3 — Incorrect fast-pointer condition

Dangerous:

```
while fast:
    fast = fast.next.next
```

because `fast.next` might be `None`.

Safe:

```
while fast.next and fast.next.next:
```

for this implementation.

Mistake 4 — Updating pointers before saving them

Wrong conceptual order:

```
change pointer
↓
try to remember old pointer
```

Correct:

```
save old pointer
↓
change pointer
```

21. Linked-List Pattern Cheat Sheet

Keep these patterns mentally available.

Find middle

```
slow = fast = head

while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
```

Reverse list

```
prev = None
curr = head

while curr:
    next_node = curr.next
    curr.next = prev
    prev = curr
    curr = next_node
```

Detect cycle

```
slow = fast = head

while fast and fast.next:
    slow = slow.next
    fast = fast.next.next

    if slow is fast:
        return True
```

Find kth from end

Create a gap:

```
fast moves k steps first

then:
slow moves 1
fast moves 1
```

When fast reaches the end, slow is k positions behind.

22. Interview Recognition Framework

When you receive a linked-list question, mentally ask:

1. Do I need the middle?
→ fast/slow
2. Do I need nodes from the end?
→ reverse / two-pointer gap
3. Do I need backward traversal?
→ reverse part of the list
4. Could there be a loop?
→ Floyd fast/slow
5. Am I deleting near the head?
→ dummy node
6. Am I changing next pointers?
→ save next before mutation
7. Can I solve with $O(1)$ extra space?
→ pointer manipulation

For today's medium problem, the recognition chain was:

```
Need front + back alternation
  ↓
can't traverse backward efficiently
  ↓
find middle
  ↓
reverse second half
  ↓
merge alternating nodes
  ↓
O(n) time / O(1) space
```

That decomposition—turning one seemingly difficult linked-list problem into **three known pointer patterns**—is the skill worth retaining.